

AIAC-ASSIGNMENT-11.2

NAME:N.Trinandini

BATCH : 32

HNO:2303A52419

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
```

```
    pass
```

Expected Output:

- A functional stack implementation with all required methods and docstrings.

```
# generate a python code to demonstrate stack and its operations
class Stack:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return len(self.items) == 0

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        else:
            raise IndexError("pop from empty stack")

    def peek(self):
        if not self.is_empty():
            return self.items[-1]
        else:
            raise IndexError("peek from empty stack")
```

```

    def size(self):
        return len(self.items)

# example
stack = Stack()
stack.push(1)
stack.push(2)
stack.push(3)
print(stack.peek()) # Output: 3
print(stack.size()) # Output: 3
print(stack.pop())  # Output: 3
print(stack.pop())  # Output: 2
print(stack.is_empty()) # Output: False

```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
    pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

```

# generate a python code to demonstrate queue and its operations
class Queue:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return len(self.items) == 0

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        else:
            raise IndexError("dequeue from empty queue")

    def peek(self):
        if not self.is_empty():
            return self.items[0]

```

```

        else:
            raise IndexError("peek from empty queue")

    def size(self):
        return len(self.items)
#example
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
queue.enqueue(3)
print(queue.peek()) # Output: 1
print(queue.size()) # Output: 3
print(queue.dequeue()) # Output: 1
print(queue.dequeue()) # Output: 2
print(queue.is_empty()) # Output: False

```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
```

```
    pass
```

```
class LinkedList:
```

```
    pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

```

# generate a python code to demonstrate linked list and its operations
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None

    def is_empty(self):
        return self.head is None

```

```

def append(self, data):
    new_node = Node(data)
    if self.is_empty():
        self.head = new_node
        return
    last_node = self.head
    while last_node.next:
        last_node = last_node.next
    last_node.next = new_node

def delete(self, key):
    current_node = self.head
    previous_node = None
    while current_node and current_node.data != key:
        previous_node = current_node
        current_node = current_node.next
    if current_node is None:
        return # Key not found
    if previous_node is None:
        self.head = current_node.next # Deleting the head node
    else:
        previous_node.next = current_node.next # Bypass the node to
delete

def search(self, key):
    current_node = self.head
    while current_node:
        if current_node.data == key:
            return True
        current_node = current_node.next
    return False

def size(self):
    count = 0
    current_node = self.head
    while current_node:
        count += 1
        current_node = current_node.next
    return count

# example
linked_list = LinkedList()
linked_list.append(1)
linked_list.append(2)
linked_list.append(3)
print(linked_list.search(2)) # Output: True
print(linked_list.size()) # Output: 3
linked_list.delete(2)

```

```
print(linked_list.search(2)) # Output: False
print(linked_list.size()) # Output: 2
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

```
class BST:
```

```
    pass
```

Expected Output:

- BST implementation with recursive insert and traversal methods.

```
# generate a python code to demonstrate binary search tree and its operations
class TreeNode:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, key):
        if self.root is None:
            self.root = TreeNode(key)
        else:
            self._insert_recursively(self.root, key)

    def _insert_recursively(self, node, key):
        if key < node.val:
            if node.left is None:
                node.left = TreeNode(key)
            else:
                self._insert_recursively(node.left, key)
        else:
            if node.right is None:
                node.right = TreeNode(key)
            else:
                self._insert_recursively(node.right, key)

    def search(self, key):
        return self._search_recursively(self.root, key)
```

```

def _search_recursively(self, node, key):
    if node is None:
        return False
    if node.val == key:
        return True
    elif key < node.val:
        return self._search_recursively(node.left, key)
    else:
        return self._search_recursively(node.right, key)

def inorder_traversal(self):
    return self._inorder_traversal_recursively(self.root)

def _inorder_traversal_recursively(self, node):
    res = []
    if node:
        res = self._inorder_traversal_recursively(node.left)
        res.append(node.val)
        res = res + self._inorder_traversal_recursively(node.right)
    return res

# example
bst = BinarySearchTree()
bst.insert(5)
bst.insert(3)
bst.insert(7)
print(bst.search(3)) # Output: True
print(bst.search(4)) # Output: False
print(bst.inorder_traversal()) # Output: [3, 5, 7]

```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
```

```
    pass
```

Expected Output:

- Collision handling using chaining, with well-commented methods.

```

# generate a python code to demonstrate hash table and its operations
class HashTable:
    def __init__(self):
        self.table = {}

```

```

def insert(self, key, value):
    self.table[key] = value

def delete(self, key):
    if key in self.table:
        del self.table[key]

def search(self, key):
    return self.table.get(key, None)

def size(self):
    return len(self.table)
# example
hash_table = HashTable()
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print(hash_table.search("name")) # Output: Alice
print(hash_table.size()) # Output: 2
hash_table.delete("age")
print(hash_table.search("age")) # Output: None
print(hash_table.size()) # Output: 1

```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```

class Graph:
    pass

```

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

```

# generate a python code to demonstrate graph and its operations
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []

```

```

        self.graph[u].append(v)
        self.graph[v].append(u) # For undirected graph

    def bfs(self, start):
        visited = set()
        queue = [start]
        while queue:
            vertex = queue.pop(0)
            if vertex not in visited:
                print(vertex, end=' ')
                visited.add(vertex)
                queue.extend(set(self.graph[vertex]) - visited)

    def dfs(self, start, visited=None):
        if visited is None:
            visited = set()
        if start not in visited:
            print(start, end=' ')
            visited.add(start)
            for neighbor in self.graph[start]:
                self.dfs(neighbor, visited)

# example
graph = Graph()
graph.add_edge(0, 1)
graph.add_edge(0, 2)
graph.add_edge(1, 2)
graph.add_edge(2, 0)
graph.add_edge(2, 3)
print("Breadth-First Search starting from vertex 2:")
graph.bfs(2) # Output: 2 0 3 1
print("\nDepth-First Search starting from vertex 2:")
graph.dfs(2) # Output: 2 0 1 3

```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's `heapq` module.

Sample Input Code:

```

class PriorityQueue:

    pass

```

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

```
# generate a python code to demonstrate priority queue and its operations
import heapq
class PriorityQueue:
    def __init__(self):
        self.elements = []

    def is_empty(self):
        return not self.elements

    def put(self, item, priority):
        heapq.heappush(self.elements, (priority, item))

    def get(self):
        return heapq.heappop(self.elements)[1]

    def peek(self):
        return self.elements[0][1] if self.elements else None

    def size(self):
        return len(self.elements)
# example
priority_queue = PriorityQueue()
priority_queue.put("task1", 2)
priority_queue.put("task2", 1)
print(priority_queue.peek()) # Output: task2
print(priority_queue.size()) # Output: 2
print(priority_queue.get()) # Output: task2
print(priority_queue.get()) # Output: task1
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

```
class DequeDS:

    pass
```

Expected Output:

- Insert and remove from both ends with docstrings.

```
# generate a python code to demonstrate deque and its operations
```

```
class Deque:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return len(self.items) == 0

    def add_front(self, item):
        self.items.insert(0, item)

    def add_rear(self, item):
        self.items.append(item)

    def remove_front(self):
        if not self.is_empty():
            return self.items.pop(0)
        else:
            raise IndexError("remove from empty deque")

    def remove_rear(self):
        if not self.is_empty():
            return self.items.pop()
        else:
            raise IndexError("remove from empty deque")

    def peek_front(self):
        if not self.is_empty():
            return self.items[0]
        else:
            raise IndexError("peek from empty deque")

    def peek_rear(self):
        if not self.is_empty():
            return self.items[-1]
        else:
            raise IndexError("peek from empty deque")

    def size(self):
        return len(self.items)

# example
deque = Deque()
deque.add_rear(1)
deque.add_rear(2)
deque.add_front(0)
print(deque.peek_front()) # Output: 0
print(deque.peek_rear()) # Output: 2
print(deque.size()) # Output: 3
print(deque.remove_front()) # Output: 0
```

```
print(deque.remove_rear()) # Output: 2
print(deque.is_empty())   # Output: False
```

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.

Feature	Data Structure	Justification
Student Attendance Tracking	Queue	Students enter and exit in order. Queue follows FIFO which perfectly matches attendance logging order.
Event Registration System	Hash Table	Participants can be searched quickly using ID or name. Hash tables provide constant-time lookup.
Library Book Borrowing	Linked List	Books can be added or removed dynamically. Linked list supports flexible insertions and deletions.
Bus Scheduling System	Graph	Bus routes and stops form network connections. Graph represents these relationships efficiently.
Cafeteria Order Queue	Queue	Orders are served in the order students arrive. Queue ensures FIFO processing.

- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
# generate a python code for bus scheduling system using graph data structure
class Graph:
    def __init__(self):
        self.graph = {}
    def add_edge(self, u, v):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append(v)
        self.graph[v].append(u) # For undirected graph
    def bfs(self, start):
        visited = set()
        queue = [start]
        while queue:
```

```

        vertex = queue.pop(0)
        if vertex not in visited:
            print(vertex, end=' ')
            visited.add(vertex)
            queue.extend(set(self.graph[vertex]) - visited)
def dfs(self, start, visited=None):
    if visited is None:
        visited = set()
    if start not in visited:
        print(start, end=' ')
        visited.add(start)
        for neighbor in self.graph[start]:
            self.dfs(neighbor, visited)
# example
bus_schedule = Graph()
bus_schedule.add_edge("Stop A", "Stop B")
bus_schedule.add_edge("Stop A", "Stop C")
bus_schedule.add_edge("Stop B", "Stop D")
bus_schedule.add_edge("Stop C", "Stop D")
print("Bus route from Stop A:")
bus_schedule.bfs("Stop A") # Output: Stop A Stop B Stop C Stop D
print("\nBus route from Stop A (DFS):")
bus_schedule.dfs("Stop A") # Output: Stop A Stop B Stop D Stop C

```

Task Description #10: Smart Hospital Management System – Data

Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from

the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque
- Justify your choice in 2–3 sentences per feature.

Feature	Data Structure	Justification
Patient Check-In	Queue	Patients are treated in order of arrival.
Emergency Case Handling	Priority Queue	Critical patients must be treated first.
Medical Records	Hash Table	Fast retrieval using patient ID.
Doctor Appointment Scheduling	BST	Appointments sorted by time efficiently.
Hospital Navigation	Graph	Wards and rooms connected like nodes in a network.

- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

#generate a python code for hash table implementation for medical records storage

```
class HashTable:
    def __init__(self, size=100):
        self.size = size
        self.table = [None] * self.size

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash(key)
        if self.table[index] is None:
            self.table[index] = [(key, value)]
        else:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    self.table[index][i] = (key, value) # Update existing key
                    return
            self.table[index].append((key, value)) # Add new key-value pair

    def get(self, key):
        index = self._hash(key)
        if self.table[index] is not None:
            for k, v in self.table[index]:
                if k == key:
                    return v
        return None # Key not found

    def delete(self, key):
        index = self._hash(key)
        if self.table[index] is not None:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    del self.table[index][i] # Remove the key-value pair
                    return True
        return False # Key not found
```

```
        return False # key not found
# Example usage
if __name__ == "__main__":
    medical_records = HashTable()
    medical_records.insert("patient_001", {"name": "John Doe", "age": 30, "condition": "Flu"})
    medical_records.insert("patient_002", {"name": "Jane Smith", "age": 25, "condition": "Cold"})

    print(medical_records.get("patient_001")) # Output: {'name': 'John Doe', 'age': 30, 'condition': 'Flu'}
    print(medical_records.get("patient_002")) # Output: {'name': 'Jane Smith', 'age': 25, 'condition': 'Cold'}

    medical_records.delete("patient_001")
    print(medical_records.get("patient_001")) # Output: None
```

```
{'name': 'John Doe', 'age': 30, 'condition': 'Flu'}
{'name': 'Jane Smith', 'age': 25, 'condition': 'Cold'}
None
```

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.

Feature	Data Structure	Justification
Traffic Signal Queue	Queue	Vehicles wait in arrival order.
Emergency Vehicle Handling	Priority Queue	Ambulances/fire trucks must move first.
Vehicle Registration Lookup	Hash Table	Fast vehicle lookup using registration number.

Feature	Data Structure	Justification
Road Network Mapping	Graph	Roads and intersections form network connections.
Parking Slot Availability	Hash Table	Quickly check free or occupied slots.

- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
#generate a python code for priority queue implementation for vehicle priority handling
class PriorityQueue:
    def __init__(self):
        self.elements = []

    def is_empty(self):
        return not self.elements

    def put(self, item, priority):
        self.elements.append((priority, item))
        self.elements.sort(key=lambda x: x[0]) # Sort by priority

    def get(self):
        if not self.is_empty():
            return self.elements.pop(0)[1] # Return the item with the highest priority
        raise IndexError("Priority Queue is empty")

# Example usage
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.put("Vehicle A", 2) # Lower number means higher priority
    pq.put("Vehicle B", 1)
    pq.put("Vehicle C", 3)

    print(pq.get()) # Output: Vehicle B (highest priority)
    print(pq.get()) # Output: Vehicle A
    print(pq.get()) # Output: Vehicle C
    # print(pq.get()) # This will raise an error since the queue is empty
```

```
Vehicle B
Vehicle A
Vehicle C
```

Task Description #12: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:

- o Stack
- o Queue
- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque

- Justify your choice in 2–3 sentences per feature.

Feature	Data Structure Justification	
Shopping Cart	Linked List	Items can be added or removed dynamically.
Order Processing	Queue	Orders processed in order placed.

Feature	Data Structure Justification	
Top Selling Products	Priority Queue	Products ranked by sales count.
Product Search	Hash Table	Instant lookup using product ID.
Delivery Route Planning	Graph	Warehouses and delivery points form a network.

- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

#generate a python code for graph implementation for delivery route planning
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v, weight=1):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append((v, weight))
        self.graph[v].append((u, weight)) # For undirected graph

    def dijkstra(self, start):
        import heapq
        queue = [(0, start)]
        distances = {node: float('inf') for node in self.graph}
        distances[start] = 0
        while queue:
            current_distance, current_node = heapq.heappop(queue)
            if current_distance > distances[current_node]:
                continue
            for neighbor, weight in self.graph[current_node]:
                distance = current_distance + weight
                if distance < distances[neighbor]:
                    distances[neighbor] = distance
                    heapq.heappush(queue, (distance, neighbor))
        return distances

# Example usage
if __name__ == "__main__":
    g = Graph()
    g.add_edge("Warehouse", "Customer A", 5)
    g.add_edge("Warehouse", "Customer B", 10)
    g.add_edge("Customer A", "Customer B", 2)

    distances = g.dijkstra("Warehouse")
    print(distances) # Output: {'Warehouse': 0, 'Customer A': 5, 'Customer B': 7}

```

```
#generate a python code for graph implementation for delivery route planning
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v, weight=1):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append((v, weight))
        self.graph[v].append((u, weight)) # For undirected graph

    def dijkstra(self, start):
        import heapq
        queue = [(0, start)]
        distances = {node: float('inf') for node in self.graph}
        distances[start] = 0
        while queue:
            current_distance, current_node = heapq.heappop(queue)
            if current_distance > distances[current_node]:
                continue
            for neighbor, weight in self.graph[current_node]:
                distance = current_distance + weight
                if distance < distances[neighbor]:
                    distances[neighbor] = distance
                    heapq.heappush(queue, (distance, neighbor))
        return distances

# Example usage
if __name__ == "__main__":
    g = Graph()
    g.add_edge("Warehouse", "Customer A", 5)
    g.add_edge("Warehouse", "Customer B", 10)
    g.add_edge("Customer A", "Customer B", 2)

    distances = g.dijkstra("Warehouse")
    print(distances) # Output: {'Warehouse': 0, 'Customer A': 5, 'Customer B': 7}
```

```
PS C:\Users\Admin\AppData\Local\Programs\Microsoft VS
{'Warehouse': 0, 'Customer A': 5, 'Customer B': 7}
PS C:\Users\Admin\AppData\Local\Programs\Microsoft VS
```

Task Description #14: Smart Banking Management System – Data

Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.
2. Loan Approval Priority Handling – High-value or urgent loans processed first.
3. Account Information Lookup – Fast retrieval of customer details

using Account Number.

4. Transaction History Management – Maintain chronological transaction records.

5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

- For each feature, select the most appropriate data structure from the list below:

- o Stack

- o Queue

- o Priority Queue

- o Linked List

- o Binary Search Tree (BST)

- o Graph

- o Hash Table

- o Deque

- Justify your choice in 2–3 sentences per feature.

Feature	Data Structure Justification	
Customer Token System	Queue	Customers served in order of arrival.
Loan Approval Priority	Priority Queue	High-value loans processed first.
Account Lookup	Hash Table	Fast retrieval using account number.
Transaction History	Linked List	Transactions stored sequentially.
ATM Connectivity	Graph	ATMs across cities connected via network.

- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
#generate a python code for queue implementation for Customer Service Token System
class Queue:
    def __init__(self):
        self.elements = []

    def is_empty(self):
        return not self.elements

    def enqueue(self, item):
        self.elements.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.elements.pop(0) # Remove and return the first item
        raise IndexError("Queue is empty")

# Example usage
if __name__ == "__main__":
    customer_queue = Queue()
    customer_queue.enqueue("Customer 1")
    customer_queue.enqueue("Customer 2")
    customer_queue.enqueue("Customer 3")

    print(customer_queue.dequeue()) # Output: Customer 1
    print(customer_queue.dequeue()) # Output: Customer 2
    print(customer_queue.dequeue()) # Output: Customer 3
    # print(customer_queue.dequeue()) # This will raise an error since the queue is empty
```

```
Customer 1
Customer 2
Customer 3
```

Task Description #15: Online Learning Management System – Data

Structure Selection

An online education platform wants to develop a Learning Management System that handles:

1. Student Login Authentication – Quick validation of user credentials.
2. Assignment Submission Queue – Submissions processed in order of upload.
3. Top Performer Ranking – Rank students based on scores.
4. Course Prerequisite Structure – Represent subject dependencies.

5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

- Select the most appropriate data structure for each feature from the given list.
- Justify your selection in 2–3 sentences per feature.

Feature	Data Structure	Justification
Student Login Authentication	Hash Table	Quick username/password lookup.
Assignment Submission	Queue	Submissions processed in order received.
Top Performer Ranking	Priority Queue	Students ranked by highest score.
Course Prerequisites	Graph	Courses depend on other courses.
Undo Feature	Stack	Undo follows LIFO order of actions.

- Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

- Feature → Data Structure → Justification table.
- Functional Python program with proper documentation.

```
#generate a python code for undo feature in online text editor using stack data structure
class Stack:
    def __init__(self):
        self.elements = []

    def is_empty(self):
        return not self.elements

    def push(self, item):
        self.elements.append(item)

    def pop(self):
        if not self.is_empty():
            return self.elements.pop() # Remove and return the last item
        raise IndexError("Stack is empty")

# Example usage
if __name__ == "__main__":
    undo_stack = Stack()
    undo_stack.push("Type 'Hello'")
    undo_stack.push("Type 'World'")
    undo_stack.push("Delete 'World'")

    print(undo_stack.pop()) # Output: Delete 'World'
    print(undo_stack.pop()) # Output: Type 'World'
    print(undo_stack.pop()) # Output: Type 'Hello'
    # print(undo_stack.pop()) # This will raise an error since the stack is empty
```

```
Delete 'World'
Type 'World'
Type 'Hello'
```